Exercise 1: Floating point numbers

Prognostic Modelling (pea700), WiSe 2026/27

Out: week 1 · Back: week 2 · 7 points

Covers unit 1 of the script. The preparation sets up the repository through which you hand in all exercises of the semester; see the rules at its end. The preparation and all three problems are worked on in the lab; what you do not finish there is homework.

Preparation: Git, GitLab and your working environment

All solutions of this semester and the final project report are handed in through the institute's GitLab server, `https://<gitlab-address>`. You will receive an invitation by e-mail during the first lab. GitLab stores your work in a **git repository**: a folder whose complete history of changes is recorded, so that every earlier state can be restored and every change can be traced. Version control is standard in scientific programming, and you will use it for every exercise.

This preparation carries no points, but without it you cannot hand in. You may use any editor. We recommend **Visual Studio Code** (VS Code), which combines an editor, a terminal, Jupyter notebooks and git in one window; see the box at the end of this section.

a) **Set up git.** Check that git is installed with `git --version` (on Windows, install *Git for Windows* from git-scm.com). Tell git who you are; this name appears in every commit you make:

```
git config --global user.name "First Last"
git config --global user.email "your.address@uni-bonn.de"
git config --global init.defaultBranch main
```

Create an SSH key, which lets you push without typing a password, and add the **public** key to GitLab under *Preferences → SSH Keys*:

```
ssh-keygen -t ed25519 -C "your.address@uni-bonn.de"
cat ~/.ssh/id_ed25519.pub        # copy this line into GitLab
ssh -T git@<gitlab-address>      # test: GitLab greets you by name
```

Never share the private key `~/.ssh/id_ed25519`.

b) **Get your repository.** Your personal repository has already been created in the course group `<course-group>`. It is named after the initial of your first name and your last name, for example `jsmith` for Jane Smith, and is visible only to you, the lecturer and the tutors. Accept the invitation from the e-mail, sign in, and copy the SSH address from the *Code* button of the project page. Clone the repository to your computer:

```
git clone git@<gitlab-address>:<course-group>/emustermann.git
cd emustermann
```

Give it the following structure: one folder per exercise sheet, one for the project, a `README.md` with your name and matriculation number, and a `.gitignore` that keeps generated files out of the repository.

```
jsmith/
    README.md
    .gitignore
    requirements.txt
    ex01/
    ex02/
    ...
    project/
```

with the `.gitignore`

```
.venv/
__pycache__/
.ipynb_checkpoints/
```

c) **Set up Python.** Create a virtual environment inside the repository, activate it, and install the packages of this course:

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install numpy scipy matplotlib jupyter
```

Write the four package names into `requirements.txt`, one per line, so that anybody can recreate the environment with `pip install -r requirements.txt`. The environment itself (`.venv/`) is not committed: it is large, and it only works on your computer.

d) **Commit and push.** Record the current state and send it to GitLab:

```
git status                       # what has changed?
git add README.md .gitignore requirements.txt
git commit -m "Set up repository for pea700"
git push
```

Check in the browser that the files have arrived. From now on every exercise follows the same cycle: work, `git add`, `git commit`, `git push`. If you work on more than one computer, start each session with `git pull`.

Rules for handing in

- Each sheet goes into its folder `exNN/`. Code as a notebook (`.ipynb`) or script (`.py`) that runs without changes in the environment from `requirements.txt`. Derivations as PDF (scan or typeset) or in the markdown cells of the notebook.
- What is pushed to GitLab before the deadline is graded. Push early and often; there is no penalty for many commits.
- Do not commit generated or large files (environments, data files, output folders).
- The final project report goes into `project/`.

Working with VS Code (optional)

1. Install VS Code from code.visualstudio.com and, in the *Extensions* view (`Ctrl+Shift+X`), the extensions **Python** and **Jupyter** (both by Microsoft).
2. *File* → *Open Folder* and choose your repository folder.
3. Select the environment: `Ctrl+Shift+P` → *Python: Select Interpreter* → `.venv`. Notebooks then use it as their kernel.
4. The *Source Control* view (`Ctrl+Shift+G`) shows changed files. Stage them with `+`, write a message, *Commit*, then *Sync Changes* to push and pull. The integrated terminal (`Ctrl+`` ` ``) accepts all commands above.

The Python you need for this sheet

Open a notebook or a Python file in your repository folder `ex01/` and try:

```
x = 0.1 + 0.2          # compute and store the result under the name x
print(x)               # show the value of x
print("x =", x)        # show a text and then the value
print(f"{x:.20f}")     # show x with 20 digits after the decimal point
print(x == 0.3)        # == compares two values: True or False

n = 0                  # a counter
while n < 3:           # repeat the indented lines as long as n < 3
    n = n + 1          # increase the counter by 1
    print("n =", n)    # prints n = 1, n = 2, n = 3
```

`=` stores a value, `==` compares two values. The lines that belong to the `while` loop are indented by four spaces; Python checks the condition before each pass and continues after the loop as soon as it is `False`. Everything else is introduced where it is needed.

Problem 1: Python as a calculator (in the lab, 2 points)

a) Before you run anything, write down what you **expect** for each line. Then run the lines and compare.

```
print(0.1 + 0.2 == 0.3)
print(f"{0.1:.20f}")
print(1 / 3 * 3 == 1)
print(1e16 + 1 - 1e16)
print(1e16 - 1e16 + 1)
print(10.0**308 * 10)
print(1e-320 / 1e10)
```

b) Explain each result in one sentence. Use the three facts about floating point numbers from the lecture: about 16 significant digits, a largest and a smallest number, and most decimal fractions are not exactly representable.

c) How should two floating point numbers `a` and `b` be compared instead of `a == b`? Write a comparison that gives `True` for `0.1 + 0.2` and `0.3`.

Problem 2: A time loop (in the lab, 2 points)

Every prognostic model has a time loop. Here is a simple one that should advance the time from 0 to 1 in steps of 0.1:

```
t = 0.0
dt = 0.1
steps = 0
while t < 1.0:
    t = t + dt
    steps = steps + 1
print(steps, t)
```

a) How many steps do you expect? Run the code. How many steps are made, and what is the final time? Explain.

b) Repeat with `dt = 0.25`. Why does the loop now behave as expected?

c) Rewrite the loop such that it always makes exactly the intended number of steps: compute the number of steps first, loop over an integer counter `n`, and compute the time from the counter as `t = n * dt`.

Problem 3: Machine epsilon and cancellation (in the lab, 3 points)

a) Determine the machine epsilon experimentally: start with `eps = 1.0` and halve it as long as `1.0 + eps / 2 > 1.0`. Compare with the value from the lecture, $2^{-52} \approx 2.2 \times 10^{-16}$.

b) The equation $x^2 - 10^8 x + 1 = 0$ has the roots $x_{1,2} = \frac{1}{2}\left(10^8 \pm \sqrt{10^{16} - 4}\right)$. The smaller root is very close to 10^{-8}. Compute it with this formula (`math.sqrt` computes the square root). How many digits are correct, and why? Find a formula that gives the smaller root accurately. Hint: $x_1 x_2 = 1$.